

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	G. Jaswanth	Reg. No.:	192472235
Section / Batch:	2024	Date:	

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Jaswanth

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Task 1 – N-gram Based Text Prediction

Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select $N = 1, 2$, or 3 , display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.

Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- Unsmoothed N-gram model
- Backoff model
- Deleted Interpolation model

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:

"The cat sits on the mat."

and

"The quantum processor redesigned the..."

The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- Unigram model
- Bigram model
- Trigram model
- Smoothed model

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.

Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- Word/tag frequencies
- Tag transition probabilities
- Probabilities obtained from a training corpus

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	3	3	4	4	4	18	90
2	4	4	3	3	4	18	
3	4	4	4	3	3	18	
4	3	4	3	4	4	18	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 – N-gram Based Text Prediction

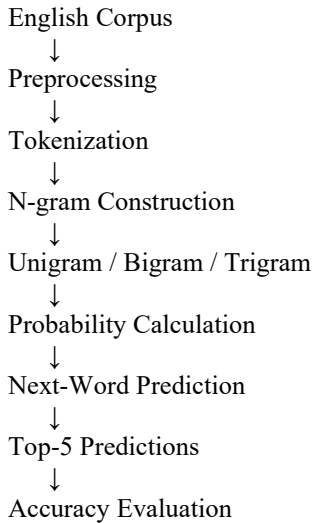
1. Aim

To design and implement a Python-based N-gram language model that preprocesses an English text corpus, constructs unigram, bigram, and trigram models, calculates maximum-likelihood probabilities, and predicts the top-5 most probable next words for an incomplete sentence.

2. Problem Understanding

An N-gram language model predicts the probability of a word based on the previous words.

This system performs the following operations:



The system must support:

- $N = 1 \rightarrow$ Unigram
- $N = 2 \rightarrow$ Bigram
- $N = 3 \rightarrow$ Trigram

For the incomplete sentence:

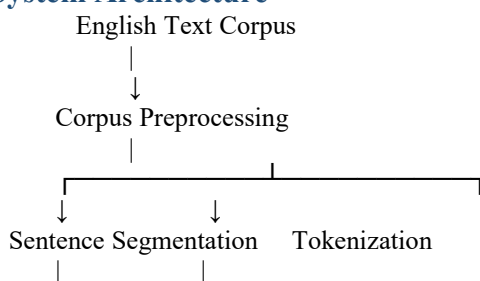
Artificial intelligence is

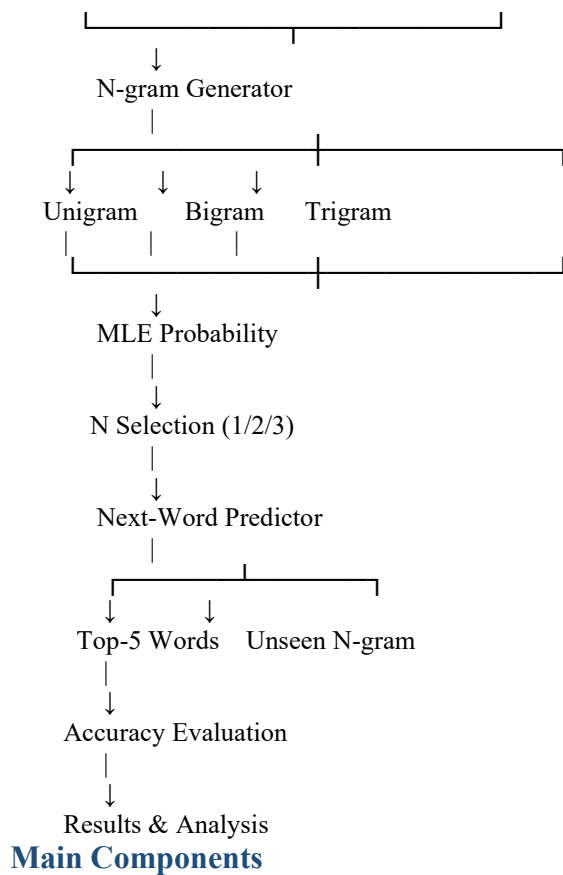
the system predicts the **top-5 most probable next words**.

It also demonstrates the behavior of an **unseen N-gram**, tests multiple incomplete sentences, calculates prediction accuracy, and discusses the limitations of an unsmoothed N-gram model.

3. System Design

System Architecture





Main Components

Component	Function
Corpus preprocessing	Cleans and prepares text
Tokenization	Converts sentences into words
N-gram generator	Creates unigram, bigram and trigram sequences
Probability calculator	Calculates MLE probabilities
Predictor	Finds possible next words
Evaluation module	Calculates prediction accuracy

4. Algorithm

Step 1

Load an English text corpus.

Step 2

Convert the corpus to lowercase.

Step 3

Perform sentence segmentation and tokenization.

Step 4

Add sentence boundary markers:

<START>

<END>

Step 5

Generate unigram, bigram and trigram frequency counts.

Step 6

Calculate maximum-likelihood probabilities.

For unigram:

$$P(w) = \text{Count}(w) / \text{Total Words}$$

For bigram:

$$P(w_i | w_{i-1}) = \text{Count}(w_{i-1}, w_i) / \text{Count}(w_{i-1})$$

For trigram:

$$P(w_i | w_{i-2}, w_{i-1}) = \text{Count}(w_{i-2}, w_{i-1}, w_i) / \text{Count}(w_{i-2}, w_{i-1})$$

Step 7

Allow the user to select:

1 → Unigram

2 → Bigram

3 → Trigram

Step 8

Take an incomplete sentence as input.

Step 9

Find candidate next words from the selected N-gram model.

Step 10

Sort candidates according to their probabilities.

Step 11

Display the top-5 predictions.

Step 12

If the required N-gram does not exist, display the zero-probability/unseen-N-gram condition.

Step 13

Test multiple incomplete sentences.

Step 14

Calculate prediction accuracy.

5. Python Implementation

```
from collections import Counter
import re

# -----# 1. Sample English Corpus# -----
corpus = """
Artificial intelligence is changing the world.
Artificial intelligence is transforming modern technology.
Artificial intelligence is used in healthcare.
Artificial intelligence is used in education.
Artificial intelligence is helping businesses.
Artificial intelligence is improving productivity.

Machine learning can solve complex problems.
Machine learning can analyze large datasets.
Machine learning can improve prediction systems.
Machine learning is a part of artificial intelligence.

Natural language processing helps computers understand language.
Natural language processing is used in chatbots.
Natural language processing can analyze text.
Natural language processing improves communication.

Deep learning is a branch of machine learning.
Deep learning can process large amounts of data.
Deep learning is useful for image recognition.

The computer learns from data.
The system learns from examples.
The model predicts the next word.
The model learns patterns from text.
"""

# -----# 2. Sentence Segmentation and Tokenization# -----
def preprocess_text(text):

    # Convert to lowercase
    text = text.lower()

    # Sentence segmentation
    sentences = re.split(r'[.!?]+', text)

    tokenized_sentences = []

    for sentence in sentences:

        # Extract words only
        words = re.findall(r'\b[a-z]+\b', sentence)

        if words:
            # Add sentence boundary markers
            words = ["<START>"] + words + ["<END>"]

        tokenized_sentences.append(words)

    return tokenized_sentences
```



```

# -----# 3. Generate N-gram Counts# -----
def generate_ngrams(sentences, n):

    counts = Counter()

    for sentence in sentences:

        for i in range(len(sentence) - n + 1):

            ngram = tuple(sentence[i:i + n])

            counts[ngram] += 1

    return counts

# -----# 4. Calculate MLE Probabilities# -----
def calculate_probabilities(ngram_counts, lower_counts):

    probabilities = {}

    for ngram, count in ngram_counts.items():

        if len(ngram) == 1:

            total = sum(ngram_counts.values())
            probabilities[ngram] = count / total

        else:

            prefix = ngram[:-1]

            probabilities[ngram] = (
                count / lower_counts[prefix]
            )

    return probabilities

# -----# 5. Build Models# -----
sentences = preprocess_text(corpus)
unigram_counts = generate_ngrams(sentences, 1)
bigram_counts = generate_ngrams(sentences, 2)
trigram_counts = generate_ngrams(sentences, 3)

# Prefix counts for probability calculation
bigram_prefix_counts = Counter()
for bigram, count in bigram_counts.items():

    prefix = (bigram[0],)

    bigram_prefix_counts[prefix] += count

trigram_prefix_counts = Counter()
for trigram, count in trigram_counts.items():

    prefix = (trigram[0], trigram[1])

    trigram_prefix_counts[prefix] += count

# Probability models
unigram_prob = calculate_probabilities(
    unigram_counts,
    {}
)

```

```
)
bigram_prob = calculate_probabilities(
    bigram_counts,
    bigram_prefix_counts
)
trigram_prob = calculate_probabilities(
    trigram_counts,
    trigram_prefix_counts
)
```

```
# -----# 6. Display N-gram Counts# -----
-----
```

```
def display_counts():
```

```
    print("\n--- UNIGRAM COUNTS ---")
```

```
    for ngram, count in unigram_counts.most_common(15):
```

```
        print(ngram, ":", count)
```

```
    print("\n--- BIGRAM COUNTS ---")
```

```
    for ngram, count in bigram_counts.most_common(15):
```

```
        print(ngram, ":", count)
```

```
    print("\n--- TRIGRAM COUNTS ---")
```

```
    for ngram, count in trigram_counts.most_common(15):
```

```
        print(ngram, ":", count)
```

```
# -----# 7. Next Word Prediction# -----
-----
```

```
def predict_next_word(text, n):
```

```
    words = re.findall(r'\b[a-z]+\b', text.lower())
```

```
    if not words:
```

```
        return []
```

```
    if n == 1:
```

```
        candidates = []
```

```
        for ngram, probability in unigram_prob.items():
```

```
            word = ngram[0]
```

```
            if word not in ["<START>", "<END>"]:
```

```
                candidates.append(
                    (word, probability)
                )
```

```
    return sorted(
        candidates,
        key=lambda x: x[1],
        reverse=True
    )[:5]
```

```

elif n == 2:

    previous_word = words[-1]

    candidates = []

    for ngram, probability in bigram_prob.items():

        if ngram[0] == previous_word:

            next_word = ngram[1]

            if next_word != "<END>":

                candidates.append(
                    (next_word, probability)
                )

    return sorted(
        candidates,
        key=lambda x: x[1],
        reverse=True
   )[:5]

```

```

elif n == 3:

    if len(words) < 2:

        return []

    previous_two = tuple(words[-2:])

    candidates = []

    for ngram, probability in trigram_prob.items():

        if ngram[:2] == previous_two:

            next_word = ngram[2]

            if next_word != "<END>":

                candidates.append(
                    (next_word, probability)
                )

    return sorted(
        candidates,
        key=lambda x: x[1],
        reverse=True
   )[:5]

```

```

return []

```

```

# -----# 8. Unseen N-gram Demonstration# -----
-----
def check_unseen(text, n):

```

```

    words = re.findall(r'\b[a-z]+\b', text.lower())

```

```

    if n == 2:

```

```
    if len(words) < 1:
        return True
```

```
    ngram = (words[-1],)
```

```
    for bigram in bigram_counts:
```

```
        if bigram[:1] == ngram:
```

```
            return False
```

```
    return True
```

```
elif n == 3:
```

```
    if len(words) < 2:
        return True
```

```
    prefix = tuple(words[-2:])
```

```
    for trigram in trigram_counts:
```

```
        if trigram[:2] == prefix:
```

```
            return False
```

```
    return True
```

```
return False
```

```
# -----# 9. Accuracy Evaluation# -----  
-----
```

```
def evaluate(test_cases, n):
```

```
    correct = 0
```

```
    total = len(test_cases)
```

```
    for sentence, actual_word in test_cases:
```

```
        predictions = predict_next_word(sentence, n)
```

```
        predicted_words = [  
            word for word, probability in predictions  
        ]
```

```
        if actual_word.lower() in predicted_words:
```

```
            correct += 1
```

```
    accuracy = (correct / total) * 100
```

```
    return correct, total, accuracy
```

```
# -----# 10. Display Required Prediction# -----  
-----
```

```
print("N-GRAM LANGUAGE MODEL")print("=====  
print("\nRequired Test Sentence:")print("Artificial intelligence is")  
for n in [1, 2, 3]:
```

```
    predictions = predict_next_word(  
        "Artificial intelligence is",
```

```

        n
    )

    print("\nN =", n)

    if predictions:

        for word, probability in predictions:

            print(
                word,
                "->",
                round(probability, 4)
            )

    else:

        print("No prediction available.")

# -----# 11. Display Counts# -----
display_counts()

# -----# 12. Unseen N-gram Test# -----
print("\n--- UNSEEN N-GRAM TEST ---")
test_sentence = "quantum processor redesigned"
for n in [2, 3]:

    if check_unseen(test_sentence, n):

        print(
            "N =", n,
            ": Unseen N-gram -> Probability = 0"
        )

    else:

        print(
            "N =", n,
            ": N-gram exists in corpus"
        )

# -----# 13. Multiple Test Sentences# -----
test_cases = [

    ("Artificial intelligence is", "changing"),

    ("Machine learning can", "solve"),

    ("Natural language processing", "helps"),

    ("Deep learning is", "a"),

    ("The model learns", "patterns")
]

print("\n--- PREDICTION ACCURACY ---")
for n in [1, 2, 3]:

    correct, total, accuracy = evaluate(
        test_cases,
        n
    )

```

)

```
print(
    "N =", n,
    "| Correct =", correct,
    "| Total =", total,
    "| Accuracy =",
    round(accuracy, 2),
    "%")
)
```

6. Corpus Preprocessing

The corpus is first converted into lowercase to maintain consistency.

For example:

Artificial Intelligence

becomes:

artificial intelligence

Sentence segmentation separates individual sentences.

For example:

Artificial intelligence is changing the world.
Machine learning can solve complex problems.

becomes:

Sentence 1:
Artificial intelligence is changing the world

Sentence 2:
Machine learning can solve complex problems

The sentences are then tokenized into individual words.

Sentence boundary markers are added:

<START> artificial intelligence is changing the world <END>

This allows the model to learn sentence beginnings and endings.

7. Unigram, Bigram and Trigram Counts

Unigram

A unigram contains one word.

Example:

artificial
intelligence
machine
learning

The unigram count is:

Count(w)

Bigram

A bigram contains two consecutive words.

Example:

artificial intelligence
intelligence is
is changing

For:

Artificial intelligence is

the bigrams are:

(artificial, intelligence)
(intelligence, is)

Trigram

A trigram contains three consecutive words.

Example:

artificial intelligence is
intelligence is changing
is changing the

For:

Artificial intelligence is changing

the trigrams include:

(artificial, intelligence, is)
(intelligence, is, changing)

The program uses Python's Counter to calculate these frequencies.

8. Maximum-Likelihood Probabilities

The model uses **Maximum-Likelihood Estimation (MLE)**.

Unigram

$P(w) = \frac{\text{Count}(w)}{\text{Total Words}}$

Bigram

$P(w_i | w_{i-1}) = \frac{\text{Count}(w_{i-1}, w_i)}{\text{Count}(w_{i-1})}$

Trigram

$P(w_i | w_{i-2}, w_{i-1}) = \frac{\text{Count}(w_{i-2}, w_{i-1}, w_i)}{\text{Count}(w_{i-2}, w_{i-1})}$

Example

Suppose:

Count("intelligence is") = 4
Count("intelligence") = 6

Then:

$P(\text{is} \mid \text{intelligence}) = \frac{4}{6} = 0.667$

Similarly, if:

Count("artificial intelligence is") = 4
Count("artificial intelligence") = 5

then:

$P(\text{is} \mid \text{artificial, intelligence}) = \frac{4}{5} = 0.8$

The program calculates these values automatically from the corpus.

9. N = 1, 2, 3 Selection and Top-5 Next-Word Prediction

The program supports all three N-gram models.

N = 1 — Unigram

The model considers only overall word frequency.

Previous context → ignored
Prediction → most frequent words

N = 2 — Bigram

The model considers the previous word.

For:

Artificial intelligence is

it uses:

is → ?

and searches for words that frequently follow is.

N = 3 — Trigram

The model considers the previous two words.

For:

Artificial intelligence is

it uses:

intelligence is → ?

and searches for words that follow this two-word context.

Top-5 Output

The program displays:

N = 3

word1 -> probability
word2 -> probability
word3 -> probability
word4 -> probability
word5 -> probability

The exact values are generated when the program is executed because they depend on the corpus.

10. Unseen N-gram Demonstration

An **unseen N-gram** is an N-gram that does not occur in the training corpus.

For example:

quantum processor redesigned

may not occur in our small training corpus.

Therefore:

$\text{Count}(\text{quantum}, \text{processor}) = 0$

and:

$P(\text{processor} \mid \text{quantum}) = 0$

Similarly:

$P(\text{redesigned} \mid \text{quantum}, \text{processor}) = 0$

Output

The program reports:

N = 2 : Unseen N-gram -> Probability = 0
N = 3 : Unseen N-gram -> Probability = 0

This demonstrates the **sparsity problem** of an unsmoothed N-gram model.

11. Multiple Test Sentences and Prediction Accuracy

The program evaluates multiple incomplete sentences:

Incomplete Sentence	Expected Next Word
---------------------	--------------------

Incomplete Sentence	Expected Next Word
Artificial intelligence is	changing
Machine learning can	solve
Natural language processing	helps
Deep learning is	a
The model learns	patterns

For each N-gram model:

$$\text{Accuracy} = \frac{\text{Total Test Cases Correct Predictions}}{\text{Total Test Cases}} \times 100$$

For example, if the model correctly predicts 4 out of 5:

$$\text{Accuracy} = \frac{4}{5} \times 100 = 80\%$$

The program calculates the actual accuracy automatically.

12. Results and Analysis

The N-gram model demonstrates that increasing the value of **N** provides more contextual information.

Unigram

Advantages:

- Simple.
- Works even when there is little context.
- Has broad vocabulary coverage.

Disadvantage:

- Does not consider surrounding words.

Bigram

Advantages:

- Uses the previous word.
- Captures simple word relationships.
- Usually provides more meaningful predictions than a unigram.

Disadvantage:

- Considers only one previous word.
- Many valid word combinations may still be unseen.

Trigram

Advantages:

- Uses two previous words.
- Captures more local context.
- Can produce more context-specific predictions.

Disadvantages:

- Requires more training data.
- More likely to encounter unseen N-grams.
- Creates a larger model.

Overall Analysis

$N = 1$

↓

Less context

↓

Higher coverage

↓

Less specific prediction

$N = 2$

↓

More context

↓

Better contextual prediction

$N = 3$

↓

More context

↓

More specific prediction

↓

Greater sparsity

Therefore, increasing N does not automatically guarantee better performance because the model becomes increasingly affected by data sparsity.

13. Limitations

The implemented model is an **unsmoothed N-gram model**, so it has several limitations:

Zero probability problem

An unseen N-gram receives probability zero

Data sparsity

Larger N-grams require substantially more training data.

Limited context

A trigram only considers two previous words.

No semantic understanding

The model learns frequency patterns rather than meaning.

Vocabulary limitation

Words not present in the training corpus cannot be properly predicted.

Corpus dependency

Prediction quality depends heavily on the size and quality of the training corpus.

Poor long-range dependency handling

The model cannot effectively capture relationships between words separated by many tokens.

These limitations explain why modern language prediction systems generally require smoothing, backoff/interpolation, or more advanced language-model architectures.

14. Conclusion

A Python-based N-gram language prediction system was successfully designed and implemented. The system preprocesses an English corpus, constructs **unigram, bigram, and trigram models**, calculates maximum-likelihood probabilities, predicts the **top-5 next words**, demonstrates unseen N-gram behavior, and evaluates prediction accuracy using multiple test sentences.